

Refactoring 1º fecha 2025

1_ Code Smells

- **Long Method:** Desde línea 5 a 44, el método tiene mucho código y es difícil de leer.
- **Switch Statements:** Desde línea 6 a 43, el método usa condicionales que pueden ser reemplazados por polimorfismo.
- **Imperative Loops:** Línea 11 a 13 y 30 a 32. Puede ser reemplazado por un stream.
- **Duplicate Code:** Línea 8 a 24 y 27 a 43. Código muy similar en estas líneas.

2_

Imperative Loops: Aplico Replace Loop With Pipeline.

- I. Reemplazo el bucle for de las líneas 11 a 13 y 30 a 32, por la llamada a un stream y almaceno el resultado en la variable "authors".

```
String authors = document.getAuthors().stream()  
    .collect(Collectors.joining(", "));;
```

Duplicate Code: Aplico Extract Method

- I. Creo un método llamado "crearDocumento" en la clase "ReportGenerator" y le paso como parámetro el documento.
- II. Muevo el código de la línea 8 a 14 y 27 a 33 al método "crearDocumento"
- III. Reemplazo el código repetido de la línea 8 a 14 y 27 a 33 por la llamada al método "crearDocumento".

```

12 private DocumentFile crearDocumento(Document document)
13 {
14     DocumentFile docFile = new DocumentFile();
15     docFile.setTitle(document.getTitle());
16     String authors = document.getAuthors().stream()
17         .collect(Collectors.joining(", "));
18     docFile.setAuthor(authors);
19     return docFile;
20 }
21
22 public void generateReport(Document document) {
23     if ("PDF".equals(type)) {
24         // crear documento y configurar metadatos
25         DocumentFile docFile = this.crearDocumento(document);
26         docFile.setContentType("application/pdf");
27         docFile.setPageSize("A4");
28
29         // crear exportador y setear el contenido
30         PDFExporter exporter = new PDFExporter();
31         byte[] content = exporter.generatePDFFile(document);
32         docFile.setContent(content);
33
34         // guardar el documento
35         this.saveExportedFile(docFile);
36
37     } else if ("XLS".equals(type)) {
38         // crear documento y configurar metadatos
39         DocumentFile docFile = this.crearDocumento(document);
40         docFile.setContentType("application/vnd.ms-excel");
41         docFile.setSheetName(document.getSubtitle());
42
43         // crear exportador y setear el contenido
44         ExcelWriter writer = new ExcelWriter();
45         byte[] content = writer.generateExcelFile(document);
46         docFile.setContent(content);
47
48         // guardar el documento
49         this.saveExportedFile(docFile);
50     }
51 }

```

Switch Statements: Aplico Replace Conditional With Polymorphism

- I. Creo una clase abstracta llamada "Tipo"
- II. Creo una clase concreta llamada "TipoPDF" y una clase concreta llamada "TipoXLS" que extienden de la clase abstracta "Tipo", con un método llamado "generateReport".
- III. A la clase concreta "PDF" le paso el código de las líneas 26 a 32 de la clase "ReportGenerator".

- IV. A la clase concreta “XLS” le paso el código de las líneas 40 a 46 de la clase “ReportGenerator”.

por el momento...

```
3 public class TipoPDF extends Tipo{
4     public void generateReport(Document document, DocumentFile docFile)
5     {
6         docFile.setContentType("application/pdf");
7         docFile.setPageSize("A4");
8
9         // crear exportador y setear el contenido
10        PDFExporter exporter = new PDFExporter();
11        byte[] content = exporter.generatePDFFile(document);
12        docFile.setContent(content);
13
14    }
15 }
```

```
3 public class TipoXLS extends Tipo{
4     public void generateReport(Document document, DocumentFile docFile)
5     {
6         docFile.setContentType("application/vnd.ms-excel");
7         docFile.setSheetName(document.getSubTitle());
8
9         // crear exportador y setear el contenido
10        ExcelWriter writer = new ExcelWriter();
11        byte[] content = writer.generateExcelFile(document);
12        docFile.setContent(content);
13
14    }
15 }
```

- V. Creo un método abstracto llamado “configureMetadata” en la “Tipo”, en las clases “TipoPDF” y “TipoXLS” lo implemento con las líneas 6 y 7 de sus respectivas clases.
- VI. Creo un método abstracto llamado “setContent” en la “Tipo”, en las clases “TipoPDF” y “TipoXLS” lo implemento con las líneas 10 y 11 de sus respectivas clases y elimino las variables temporales.
- VII. En las clases “TipoPDF” y “TipoXLS” reemplazo el código de las líneas por el método “configureMetadata” y el código de las líneas 10 y 11 los elimino junto con los comentarios, en la línea 12 reemplazo la variable “content” por el llamado al método “setContent”.

```

3 public class TipoPDF extends Tipo{
4     protected void configureMetadata(Document document, DocumentFile docFile)
5     {
6         docFile.setContentType("application/pdf");
7         docFile.setPageSize("A4");
8     }
9
10    @Override
11    protected byte[] setContent(Document document, DocumentFile docFile) {
12        return new PDFExporter().generatePDFFile(document);
13    }
14
15    public void generateReport(Document document, DocumentFile docFile)
16    {
17        this.configureMetadata(document, docFile);
18        docFile.setContent(this.setContent(document, docFile));
19    }
20 }

```

```

3 public class TipoXLS extends Tipo{
4     protected void configureMetadata(Document document, DocumentFile docFile)
5     {
6         docFile.setContentType("application/vnd.ms-excel");
7         docFile.setSheetName(document.getSubtitle());
8     }
9
10    @Override
11    protected byte[] setContent(Document document, DocumentFile docFile) {
12        return new ExcelWriter().generateExcelFile(document);
13    }
14
15    public void generateReport(Document document, DocumentFile docFile)
16    {
17        this.configureMetadata(document, docFile);
18        docFile.setContent(this.setContent(document, docFile));
19    }
20 }

```

VIII. Realizo un Pull Up Method a “generateReport”

```

3 public abstract class Tipo {
4     protected abstract void configureMetadata(Document document, DocumentFile docFile);
5     protected abstract byte[] setContent(Document document, DocumentFile docFile);
6
7     public void generateReport(Document document, DocumentFile docFile, TipoXLS tipoXLS) {
8         tipoXLS.configureMetadata(document, docFile);
9         docFile.setContent(tipoXLS.setContent(document, docFile));
10    }
11 }

```

IX. En la clase “ReportGenerator” modifiqué el tipo de la variable “type” de String a Tipo y actualicé el constructor para cambiar el tipo de la variable que recibe a Tipo. Modifiqué el método “generateReport” para que cree el documento llame a “this.type.generateReport” y luego guarde el documento.

```

5 public class ReportGenerator {
6     private Tipo type;
7
8     public ReportGenerator(Tipo type) {
9         this.type = type;
10    }
11
12    private DocumentFile crearDocumento(Document document)
13    {
14        DocumentFile docFile = new DocumentFile();
15        docFile.setTitle(document.getTitle());
16        String authors = document.getAuthors().stream()
17            .collect(Collectors.joining(", "));
18        docFile.setAuthor(authors);
19        return docFile;
20    }
21
22    public void generateReport(Document document) {
23        DocumentFile docFile = this.crearDocumento(document);
24        this.type.generateReport(document, docFile);
25        this.saveExportedFile(docFile);
26    }
27
28    // Método auxiliar para guardar el archivo exportado
29    private void saveExportedFile(DocumentFile docFile) {
30        System.out.println("Guardando archivo: " + docFile.getTitle() + " con tipo: " + docFile.getContentType());
31    }
32 }

```

Long Method: Aplicando los refactoring previos, se soluciono este Bad Smell

Mostrar código